

WIRE FRAMEWORK · RELEASE TYPE WALKTHROUGH

Dashboard-First.

When the dashboard drives the data model.

The release type we reach for when the client needs to see something tangible early — and the warehouse can be built backwards from the dashboards.

Developer Onboarding | Rittman Analytics

V3.7.0

CHOOSING THE RELEASE TYPE

When to choose **dashboard_first**

- 01 The headline output is a **small number of dashboards** — usually one exec dashboard or a focused mart.
- 02 The client **trusts the mock more than the model**. Sponsor signs off on the visuals first; the data model gets shaped to fit.
- 03 Source data is **well understood**, but the gold layer is greenfield. You're not building Bronze / Silver / Fivetran from scratch.
- 04 You need a **v0 demo in two to three weeks** — not the eight-to-twelve a `full_platform` build would take.

PICK BY THE DOMINANT WORK, NOT THE DOMINANT ARTIFACT

Don't choose dashboard-first when...

✗ Scope spans more than **~3 dashboards or 4 source systems**.

full_platform

✗ The hard problem is **ingestion** — Fivetran connectors, custom APIs, CDC.

pipeline_only

✗ The dashboards **already exist** in some form and you're just adding to them.

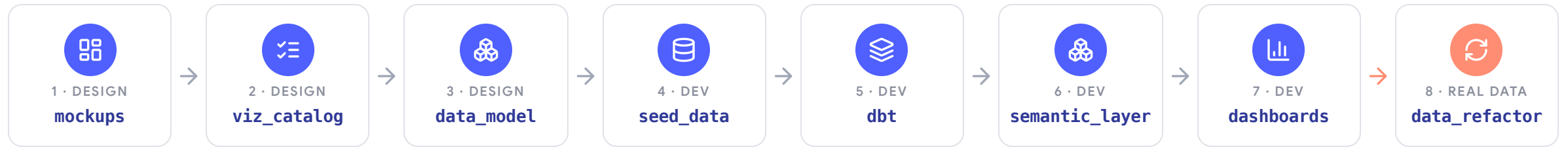
dashboard_extension

✗ The **dbt project already exists** and you're extending the warehouse.

dbt_development

THE FLOW AT A GLANCE

Mockups first, model last — the inverse of full platform.



↑ when real data lands, data_refactor rewrites staging and the dbt build re-runs — dashboards unchanged.

Mockups first, model last

The inverse of full_platform — what we need defines what we build.

Two builds

First on synthetic seed data; second after refactor swaps in real shapes.

Eight artifacts

Each with generate / validate / review where appropriate.

STEP 1 OF 8 · DESIGN

1 Mockups

COMMANDS `/wire:mockups-generate`
`/wire:mockups-review`

INPUTS SoW, stakeholder map, mockup spec.

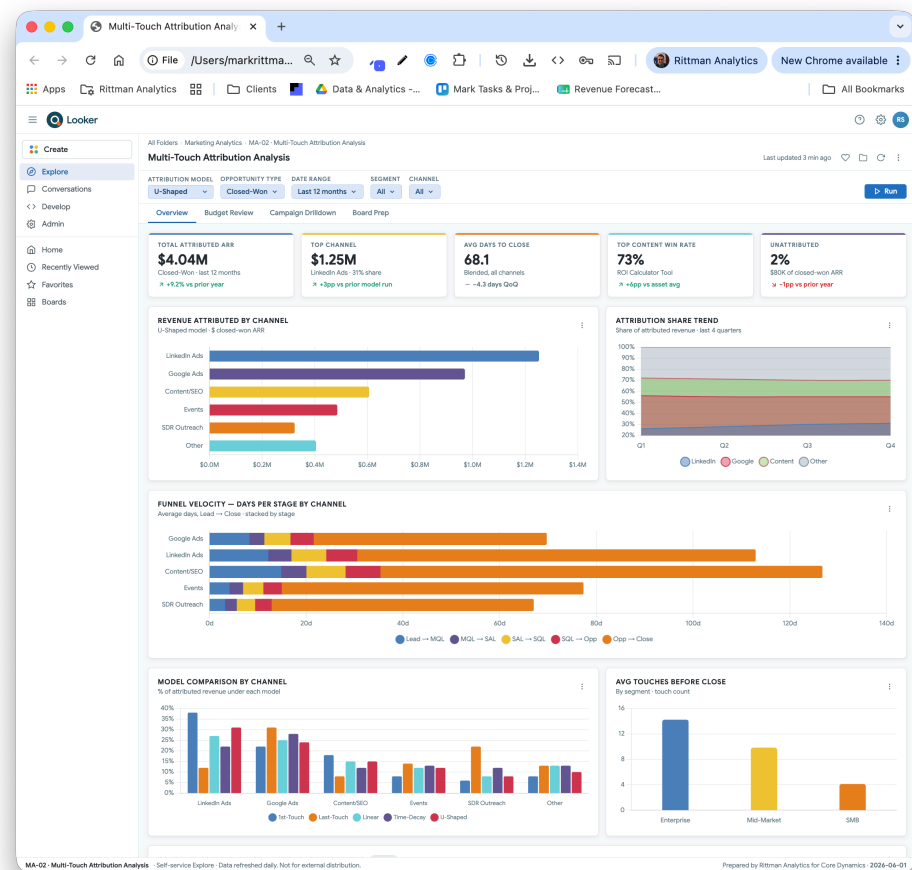
OUTPUTS `design/mockups.html` — pixel-accurate
Looker mockup via the `looker-dashboard-mockup` skill.

HITL GATE Sponsor sign-off on the visuals before any data work begins.

● WHY IT MATTERS

This is the artifact you take to the sponsor in week one.

Full Looker UI chrome — sidebar, filter pills, KPI tiles, Chart.js charts. To a sponsor in a meeting, it looks like Looker.

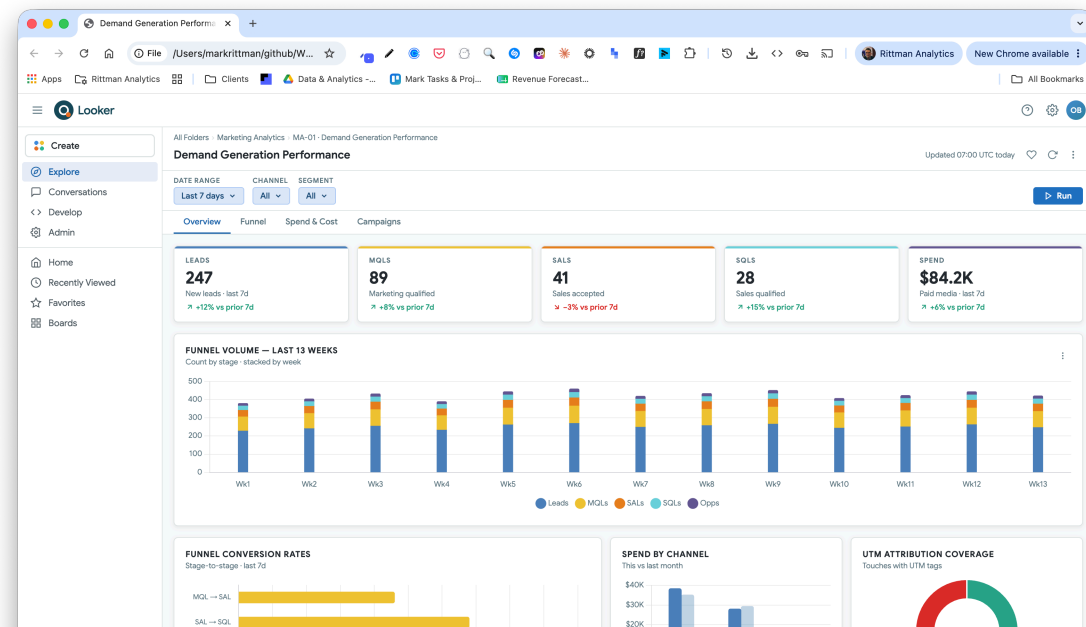


Each one generated from a brief statement of work.

Same skill, different briefs — a few sentences of SoW becomes a sponsor-ready Looker mockup before any data work starts.

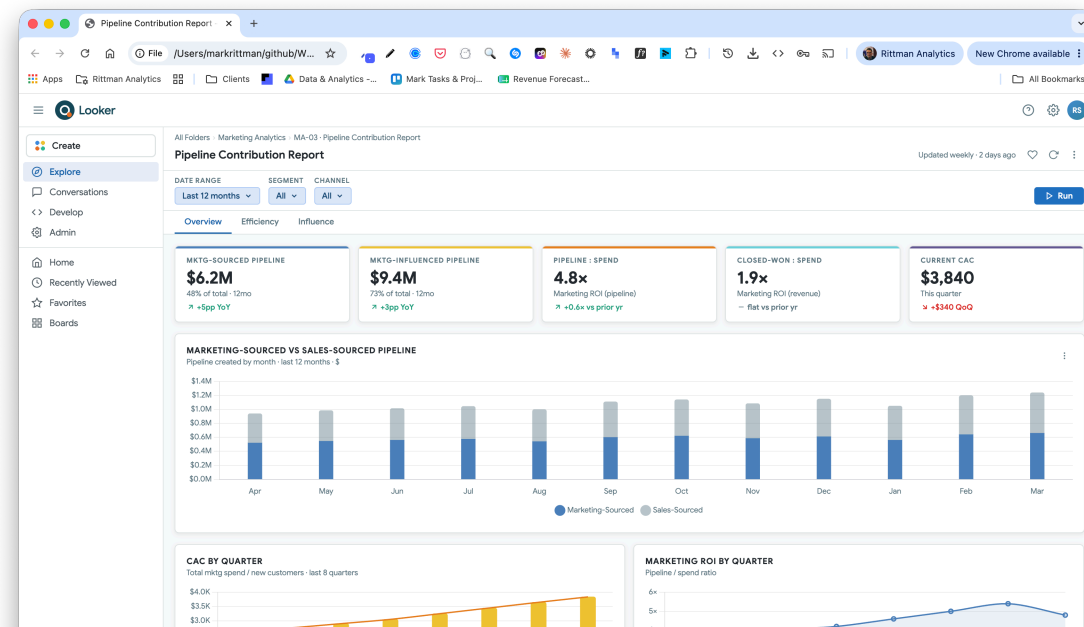
MA-01 Demand Generation

SoW: “A weekly stand-up view of funnel volume, conversion and spend by channel for the demand-gen team.”



MA-03 Pipeline Contribution

SoW: “An executive board view of marketing-sourced vs influenced pipeline, CAC and marketing ROI by quarter.”



2 Viz catalog

COMMAND /wire:viz_catalog-generate

INPUTS Approved mockups.

OUTPUTS design/viz_catalog.md — one row per chart. Measures and dimensions either come from the SoW / spec, or are **visually read by Claude Code or Gemini** straight off the chart images to extract them.

HITL GATE None — mechanical extraction.

● WHY IT MATTERS

The viz catalog is the contract the data model must satisfy.

Every chart becomes one or more rows — the source of truth that drives the model's grain.

MA-03 — Pipeline Contribution Report (Monthly Executive)



Purpose: Monthly executive view for board prep and sales/marketing alignment — marketing's contribution to pipeline and revenue, and return on marketing investment. Audience: Marcus Elwood (CEO), David Park (CRO), Rachel Summers. Refresh: weekly. Default: last 12 months.

Source models: fct_opportunity_attribution , fct_campaign_spend , stg_salesforce__opportunities

#	Visualization	Type	Measures	Dimensions	Requirement(s)
1	Marketing-Sourced Pipeline (KPI)	KPI tile	attributed_pipeline_usd (is_marketing_sourced)	(12mo rollup)	FR-009, FR-004
2	Marketing-Influenced Pipeline (KPI)	KPI tile	attributed_pipeline_usd (is_marketing_influenced)	(12mo rollup)	FR-009, FR-004
3	Pipeline : Spend (KPI)	KPI tile	pipeline_to_spend_ratio	(12mo rollup)	FR-009, FR-002, FR-004
4	Closed-Won : Spend (KPI)	KPI tile	closed_won_to_spend_ratio	(12mo rollup)	FR-009, FR-002, FR-004
5	Current CAC (KPI)	KPI tile	cac (total_spend / new_customers)	quarter	FR-009, FR-002
6	Sourced vs Sales-Sourced Pipeline	Stacked bar	attributed_pipeline_usd	month, is_marketing_sourced	FR-009, FR-004

STEP 3 OF 8 · DESIGN

3 Data model from the dashboards

COMMANDS `/wire:data_model-generate`
`-validate · -review`

INPUTS Viz catalog + source DDL.

OUTPUTS `design/data_model_specification.md` —
staging → integration → warehouse design,
with PKs, FKs, conformed dimensions,
materialisation.

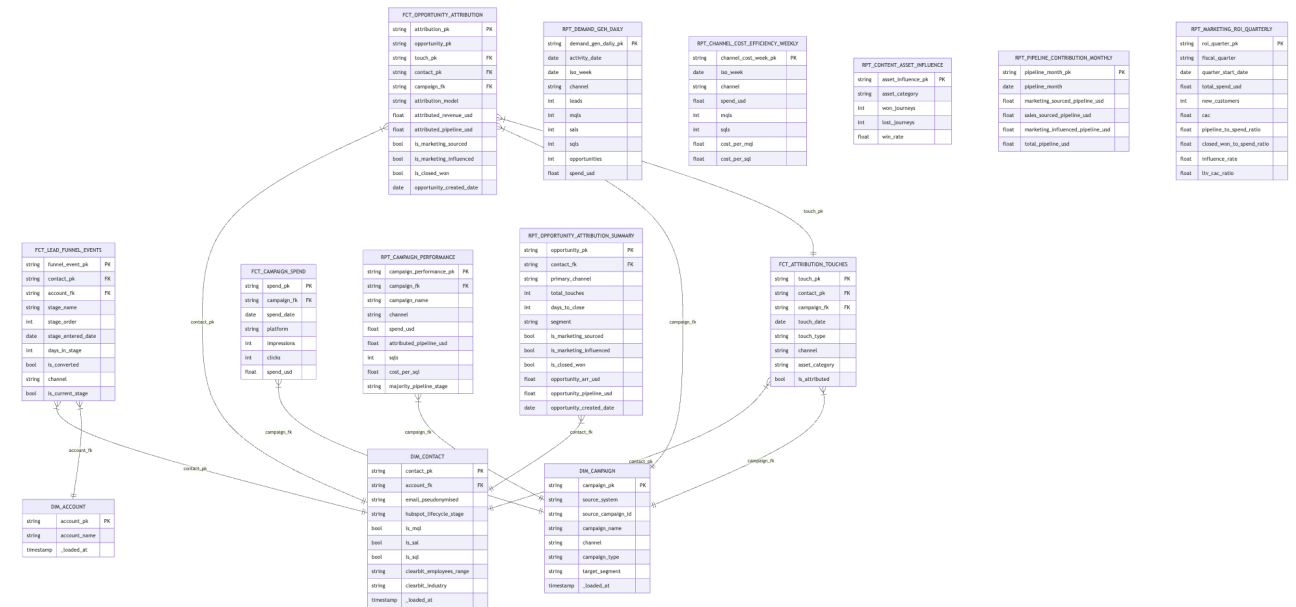
HITL GATE Analytics Engineering review.

● DIFFERENTIATOR

The model is derived from what the dashboards need.

Not top-down from entity analysis — so you'll often see a leaner model than full_platform produces.

7. Physical Data Model



Lineage not shown as FK lines (aggregate dependencies, not key joins): `rpt_demand_gen_daily` and `rpt_channel_cost_efficiency_weekly` aggregate `fact_lead_funnel_events` + `fact_campaign_spend` at channel grain; `rpt_pipeline_contribution_monthly` and `rpt_marketing_roi_quarterly` aggregate `rpt_opportunity_attribution_summary`; `rpt_content_asset_influence` joins `fact_attribution_touches` to `rpt_opportunity_attribution_summary`.

STEP 4 OF 8 · DEVELOPMENT

4 Seed data

COMMANDS `/wire:seed_data-generate`
`-validate · -review`

INPUTS Data model + a row-count target.

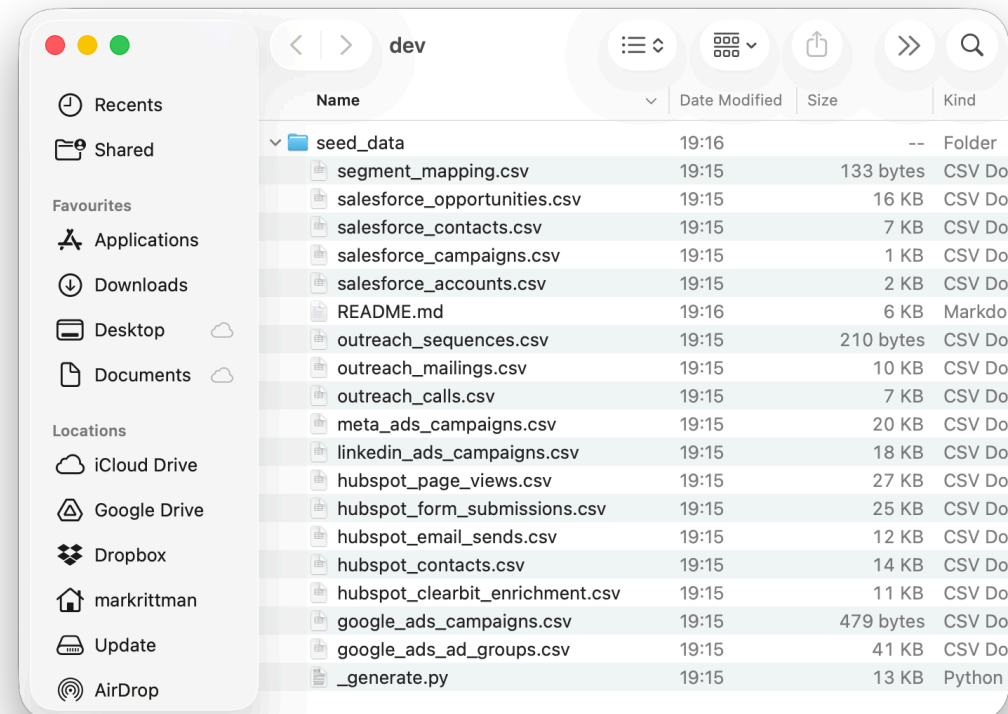
OUTPUTS Synthetic CSV seeds under `dbt/seeds/` —
deterministic and re-runnable.

HITL GATE Stakeholder spot-check — does the
synthetic data look plausible?

● WHY IT MATTERS

The prototype renders against these seeds.

Faker + a fixed RNG, so seeds are identical on every clone.
The bar: good enough for a 15-minute board demo.



STEP 5 OF 8 · DEVELOPMENT

5 dbt + first build

COMMANDS	<code>/wire:dbt-generate</code> <code>-validate · -review · dbt build</code>
INPUTS	Data model + seed data.
OUTPUTS	Staging / integration / warehouse models, all materialised; full test coverage; a working <code>warehouse.duckdb</code> (or BigQuery dataset).
HITL GATE	dbt code review.

● VALIDATION

Wire dbt-development convention checks.

`_pk, _fk, _ts` naming · test coverage · documentation. At this point you have a working dbt project on synthetic data — the next step makes it queryable through a BI tool.

STEP 6 OF 8 · DEVELOPMENT

6 Semantic layer

COMMANDS `/wire:semantic_layer-generate
-validate · -review`

INPUTS Warehouse models from dbt.

OUTPUTS LookML views, explores, and a model file in
`semantic_layer/lookml/`.

HITL GATE BI lead review.

● VALIDATION

Every `${TABLE}` . column reference is cross-checked against the dbt manifest.

No broken column references. This is where the dashboard mockup starts becoming a real Looker artifact — the viz catalog from Step 2 maps directly to explores here.

STEP 7 OF 8 • DEVELOPMENT

7 Working dashboards

COMMANDS `/wire:dashboards-generate`
`-validate` · `-review`

INPUTS Mockups + semantic layer + seed data.

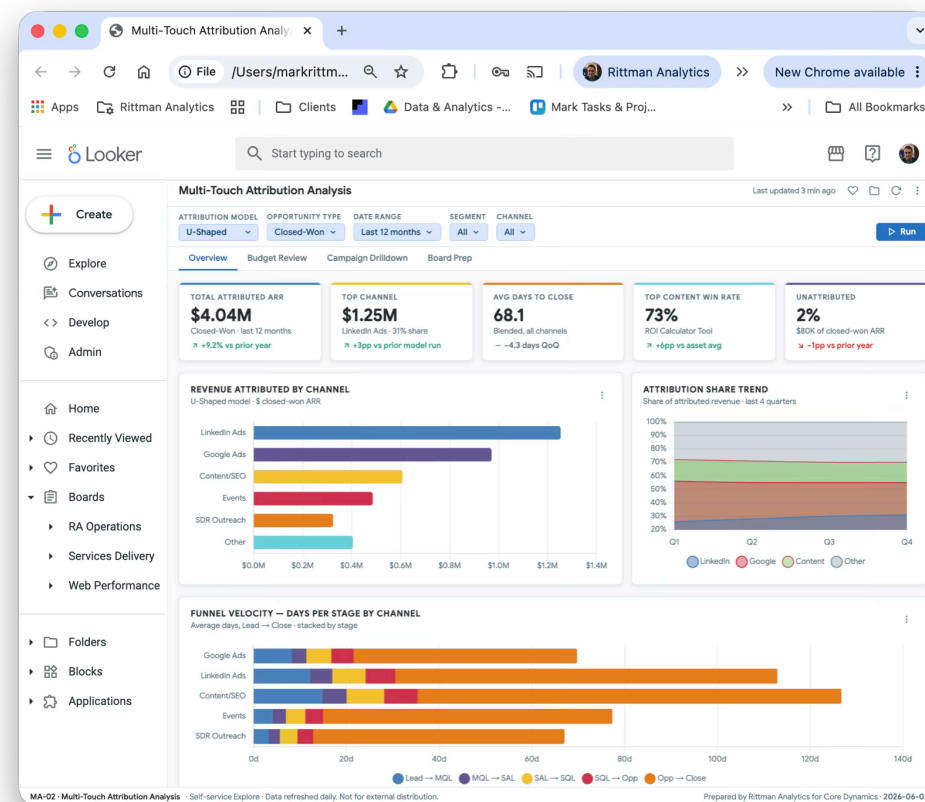
OUTPUTS `dashboards/v1.html` — same layout as the Step 1 mockup, now backed by real query results against the seeds.

HITL GATE Sponsor prototype demo.

● SECOND SPONSOR CHECKPOINT

The Step 1 mockup is now live in Looker — same charts, real numbers.

For most sponsors this is the moment it clicks: "We're not looking at a picture any more, we're looking at the thing."



STEP 8 OF 8 · REAL DATA LANDS

8 Refactor onto real data

COMMANDS `/wire:data_refactor--generate
-validate · -review`

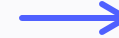
TRIGGERED BY Real client data lands — usually different column names, nested JSON, different date formats from the synthetic seeds.

OUTPUTS `dev/data_refactor_mapping.md` — explicit synthetic → real schema mapping, plus updated staging models.

THEN RE-RUN `/wire:dbt--generate --mode refactor
dbt build · re-render dashboards`



Synthetic seeds



Real client data

● THE PAYOFF

Same dashboards, real numbers, no rework.

The dashboard layout, the data model, the LookML — none of it changes. Only the staging layer is rewritten.

HUMAN-IN-THE-LOOP

Seven review gates — the sponsor only sees polished output.

#	GATE	WHO	WHAT
1	mockups-review	Sponsor	Approves the visual design
2	data_model-review	Analytics Eng	Sanity-checks model design
3	seed_data-review	Stakeholder	Confirms synthetic data realism
4	dbt-review	Code reviewer	Wire conventions + SQL quality
5	semantic_layer-review	BI lead	LookML structure + naming
6	dashboards-review	Sponsor	Live dashboard demo
7	data_refactor-review	Sponsor	Real-data go-live

■ Sponsor-facing gates — mockup, prototype, real-data demo
 ■ Internal RA gates — model, seeds, dbt, LookML

GETTING STARTED

Two commands to your first mockup.

```
/wire:new
```

```
# → choose "Dashboard-first rapid dev"
```

```
# → answer engagement / SoW / Jira questions
```

```
/wire:mockups-generate releases/01-revenue-dashboards
```

The kick-off command is release-type aware too: `/wire:kickoff-generate` produces a sponsor deck framed around the dashboard-first flow.

● LEARN IT FIRST

Watch demo 3 before your first real engagement.

The end-to-end demo lives in `wire-demos/demo3-dashboard-first/` — it runs this exact flow against the Acme Coffee Roasters synthesized client.

Full spec: `wire/specs/design/mockups/`, `wire/specs/design/viz_catalog.md`, and the per-artifact specs under `wire/specs/development/`.